舶機開発部
C言語コーディングガイド

文書番号：SVT-11B-002-2

改　訂	2021年6月23日	
発　行	2011年3月15日	
作　成	舶用機器 開発部	
承　認	審　査	作　成
開発部 2011/03/28 平岡	開発部 2011/03/28 上田　稔	舶用機器事業部 開発部 ソフトウェア 設計管理課

古野電気株式会社

[FURUNO ELECTRIC CO.,LTD. CONFIDENTIAL]

目　　次

目　　次 1

第 1 章 本書について 4
1-1. 目的 4
1-2. 適用範囲 4
1-3. 実施時期 4
1-4. 用語説明 4
1-5. 改廃 4

第 2 章 コーディングガイド 5
2-1. 可読性 5
2-1-1. continue / goto / break は濫用しない 5
2-1-2. switch 文でのフォールスルーは避けよ！ 6
2-1-3. return 文ばら撒き注意！ 7
2-1-4. 通常処理を前に！ 8
2-1-5. マジックナンバーを埋め込まない 9
2-1-6. 関連する定数を定義する時は、#define より enum を使用する 10
2-1-7. マクロ定義は“ありきたりなネーミング”にしない 11
2-2. 関数 12
2-2-1. 関数ポインタ利用で高速化 12
2-2-2. 無限ループに注意せよ！ 13
2-3. コメント 14
2-3-1. 関数ヘッダーはわかりやすく 14
2-3-2. 流用物の変更個所を明確化 15
2-3-3. コメント追加をためらうな！ 16
2-3-4. デバイス設定はコメントに！ 17
2-3-5. 未実装箇所を明確に！ 18
2-3-6. 処理時間計測結果をコメントに！ 19
2-3-7. ハードウェア依存の明示 20
2-3-8. 処理待ち時間の根拠を記載せよ！ 21
2-4. 書式 22
2-4-1. マクロは行頭から記述する！ 22
2-4-2. インデント方法の統一 23
2-4-3. 演算順序を明確にするために括弧をつける 24
2-4-4. 関数プロトタイプ宣言にも変数名を 25
2-4-5. 関数名の後にスペースを 26
2-4-6. 半角カナは使わない 27
2-4-7. 中括弧は省略しない 28
2-4-8. マクロは括弧で括る 29
2-4-9. 無限ループの記述方法 30
2-4-10. 命名規則を統一 31
2-4-11. 繰り返し処理の継続条件を条件部に明示 32
2-4-12. long 値を表すには大文字の ’L’ を使う 33
2-4-13. 中括弧はネストの始点と終点で揃える 34
2-5. 処理系 35
2-5-1. 処理系依存表現の禁止 35
2-5-2. 負数に対する割り算に注意 36
2-5-3. volatile 修飾された変数が間違ってコンパイルされることに注意 37
2-6. 条件文 38
2-6-1. 範囲チェックは最大/最小の両方で 38
2-6-2. 比較対象の明示 39
2-6-3. 条件文の中で、代入式を用いない！ 40
2-6-4. switch 文の default のスペルミスに注意 41

2-6-5.　if-else if 文は、１種類の変数に注目して表現すること 42
2-6-6.　比較対象の数値は前置きにする 43
2-6-7.　条件文内の変数インクリメントに注意 44
2-7. 変数 45
2-7-1.　union の取扱い-エンディアンフリーな表記方法 45
2-7-2.　戻り値用変数名も値の意味が連想される名前にせよ 46
2-7-3.　オート変数の使い回しは、しない！ 47
2-7-4.　変数名は、定義どおりに 48
2-7-5.　名称は仕様書から取ろう 49
2-7-6.　予約語まがいの名前を変数名に使わない 50
2-7-7.　オート変数名は単純に 51
2-7-8.　変数名からその目的が分かるように 52
2-7-9.　ビット反転、ビットシフト時の汎整数拡張 53
2-7-10.　ビットシフトの仕様はコンパイラ依存 54
2-7-11.　符号有変数と符号無変数を混在させて比較しない 55
2-7-12.　符号有から無のキャストは変数と同じ bit 長で 56
2-7-13.　比較時の汎整数拡張に注意！ 57
2-7-14.　全角文字列を扱う場合の比較に注意 58
2-7-15.　負値をシフトで割算しても期待値にならない 59
2-7-16.　int 型は 16 ビット符号付整数の範囲でのみ使用せよ 60
2-7-17.　char 型には signed、 unsigned をつけよ 61
2-7-18.　変数宣言と初期値代入について 62
2-7-19.　typedef で見やすく 63
2-7-20.　AUTO 変数のサイズはスタックを意識しろ 64
2-7-21.　非負物理量であっても、符号付き変数として宣言しよう 65
2-8. 引数 66
2-8-1.　構造体はアドレス渡しで効率化 66
2-8-2.　入力引数の const は積極的に宣言しよう 67
2-8-3.　関数の引数(またはサイズ)を多くしないこと 68
2-9. 信頼性 69
2-9-1.　関数の戻り値チェック、エラー処理を忘れずに！ 69
2-9-2.　初期化関数のコール回数に注意！ 70
2-9-3.　スタックサイズに注意！ 71
2-9-4.　排他制御用資源の初期設定値 72
2-10. #include 73
2-10-1.　ヘッダーファイルは適材適所 73
2-10-2.　ファイル名は、大文字または小文字に統一 74
2-11. #define 75
2-11-1.　配列宣言時にマジックナンバーを使うな 75
2-11-2.　#define 定義の注意点は、コメントに記載せよ 76
2-12. 保守 77
2-12-1.　#if 0 は思い切って削除を 77
2-12-2.　一時的な実装／削除の方法 78
2-12-3.　流用コードは常に最新版を 79
2-12-4.　疑わしきは使わず 80
2-12-5.　コンパイルスイッチ名は、機能名で！ 81
2-12-6.　機種依存の実装は駄目 82

第 3 章 プログラム設計時 83
3-1. 関数 83
3-1-1.　【関数切り出しの原則】関数の部品化とソースコードの再利用 83
3-1-2.　再帰関数取扱い注意！ 84
3-2. 実数演算 85
3-2-1.　収束問題 85
3-2-2.　0（ゼロ）や微小値での割り算に注意！ 86

3-2-3.　【実数演算処理速度】代替案を考えよ！ 87
3-2-4.　実数比較は幅を持たせること 88
3-2-5.　実数データの整数化 89
3-2-6.　実数精度により三角関数の計算結果が範囲を超える 90
3-3. 論理演算 91
3-3-1.　否定論理は減らそう 91
3-4. 変数 92
3-4-1.　静的変数をなるべく減らす 92
3-4-2.　変数参照の前に値は代入されているか？ 93
3-4-3.　アドレスアラインメントに注意 94
3-4-4.　タスク間通信の方法 95
3-5. 割り込み 96
3-5-1.　割込み禁止/許可の実装に注意 96
3-6. 拡張性 97
3-6-1.　データベースの情報は、なるべく最後まで持ち続ける。 97

第 1 章　本書について

1-1. 目的

本書は、ソフトウェア開発において発生した失敗事例や指摘内容をもとにしたノウハウ、哲学の「情報共有」と、ソースコードレビュー、静的解析ツールによる「指摘数削減」を目的としている。

1-2. 適用範囲

舶用機器事業部のソフトウェア開発に適用する。

1-3. 実施時期

2010 年 9 月 1 日より，運用を開始する。

1-4. 用語説明

用語	説明
運用基準:遵守	絶対に守らなければいけない項目
運用基準:推奨	開発時にソフトリーダーが方針を決める必要がある項目
運用基準:注意点	不具合に直結しないが、主に「保守」という観点から開発を進める上で参考となる項目
運用基準:ノウハウ	不具合に直結しないが、主に「生産性の向上」という観点から開発を進める上で参考となる項目

1-5. 改廃

本書を改訂する必要が生じた場合は、開発部内及び関連部署で審議し、ソフトウェア設計管理課長の承認のうえ改訂を実施する。

第 2 章　コーディングガイド

2-1. 可読性

2-1-1. continue / goto / break は濫用しない

カテゴリ	可読性-continue 文、goto 文、break 文
運用基準	推奨

【詳細説明】
一般に continue や goto 、 break 文は、処理が飛ぶのでコードの可読性を下げる。
continue、 goto、 break 文を使用せずに「判りやすく」実現できる方法があればそれを優先する。

< continue、 break >

【良くない例】

```
int    n= 0;
for(; ;){
        if( n == 2 ) {continue;}
        if( n >＝  10 ){ break;}
        process1();
        n++;
}
```

【良い例】

```
int      n;
for( n=0; n<10 ; n++ ){
        if( n != 2 ){
                process1();
        }
}
```

< goto >

【良くない例】

```
int    n = 0;
for(; ;){
        if( n == 10 ){
                goto lab;
        }
        process1();
        n++;
}
lab:  process3();
```

【良い例】

```
int    n = 0;
for( n=0; n<10 ; n++ ){
        process1();
}
Process3();
```

【備考】

2-1-2. switch 文でのフォールスルーは避けよ！

カテゴリ	可読性-switch 文
運用基準	推奨

【詳細説明】
switch 文のフォールスルー（break せず、次の case 文へ処理がまたがること）は、C/C++では許容されるが、処理の流れがわかりづらくなりがちである。

※静的解析ツール(PGRelief)の pgr0007(a 指摘)で検出可能

【良くない例】

```
switch( n ){
    case( 1 ):  process1();
    case( 2 ):  process2();
                break;
    default:    break;
}
```

【良い例】

```
switch( n ){
    case( 1 ):  process1();
                process2();
                break;
    case( 2 ):  process2();
                break;
    default:    break;
}
```

【備考】
全く同じ処理を行うケースのフォールスルーについては許容範囲とする。

```
switch( n ){
    case( 1 ):
    case( 2 ):  process1();
                process2();
                break;
    default:    break;
}
```

2-1-3. return 文ばら撒き注意！

カテゴリ	可読性-return 文
運用基準	推奨

【詳細説明】
関数内で複数の場所に return 文が散らばっているとロジックを追いにくく判りにくいため一箇所にまとめるべきである。
switch 文、for 文、 while 文内での return 文の使用は控える。
【許される例】
１） 関数のトップで例外排除のリターンがあって、正常系では、最後一箇所でリターンするなど
２） リターンがなければネストが多くなるようなケース
３）リターンがなければ、後続処理が複雑になるケース

```
#define   E_OK      (0)
#define   E_NG      (1)
```

【良くない例】

```
switch( n ){
    case( 1 ):  process1_1();
                process1_2();
                return( E_OK );
    case( 2 ):  process2_1();
                process2_2();
                if( x < 0 ){
                    process2_3();
                    return( E_NG );
                }else{
                    return( E_OK );
                }
    default:    return( E_NG );
}
```

【良い例】

```
int   flg=E_NG;
switch( n ){
    case( 1 ):  process1_1();
                process1_2();
                flg = E_OK;
                break;
    case( 2 ):  process2_1();
                process2_2();
                if( x < 0 ){
                    process2_3();
                }else{
                    flg = E_OK;
                }
                break;
    default:    break;
}
return( flg );
```

【備考】
1) return 文が複数あると割り込み禁止の解除忘れ、排他制御の資源解放忘れ、メモリ解放忘れなどを発生させる危険がある。
2) return 文が複数あるとデバッグ時の break ポイントを複数設定する必要があり、デバッグが非効率になる恐れがある。

2-1-4. 通常処理を前に！

カテゴリ	可読性-構成
運用基準	推奨

【詳細説明】
プログラム構成が、通常系処理と例外系処理に分かれるときは、通常系処理を先に記述したほうが読みやすい。（例外的な処理を後にまわすことで可読性が向上する。）

```
#define TRUE (1)
【良くない例】
judge = isTrue();
if( judge != TRUE ){  /*  例外処理  */
        exceptionProcedure();
        exit();
}else{      /*  通常処理  */
        operation1();
}

【良い例】
judge = isTrue();
if( judge == TRUE ){ /*  通常処理  */
        operation1();
}else{      /*  例外処理  */
        exceptionProcedure();
        exit();
}
```

【備考】

2-1-5. マジックナンバーを埋め込まない

カテゴリ	可読性-マクロ
運用基準	推奨

【詳細説明】
マクロ化することにより、定数の意味を明確に示すことができ、定数が複数箇所で使われているプログラムの変更時も 1 つのマクロを変更すれば済み、変更ミスが防げる。
ただしデータの大きさは、マクロではなく、sizeof を使用する。

【良くない例】

```
if( cnt == 8 ){
    .....
```

【良い例】

```
#define   MAXCNT      8
if( cnt == MAXCNT ){
    .....
```

【備考】上記の#define 以外にも enum を利用したマクロ化があり、関連する定数を定義する時は、define よりも enum を使用する方が良い。2-1-6 参照。

2-1-6. 関連する定数を定義する時は、#define より enum を使用する

カテゴリ	可読性
運用基準	推奨

【詳細説明】
関連する定数を定義するときは、関連する定数ごとにenum 型で定義する。これにより誤った使い方をした場合にコンパイラや静的解析でチェックすることが可能になる場合がある。

【悪い例】

```
#define ENGLAND 0
#define FRANCE    1
#define SUNDAY   0
#define MONDAY  1
int country, day;
…
if ( country == ENGLAND ) {
if ( day == MONDAY ) {
if ( country == SUNDAY ) { /*  ツールでチェック不可  */←正常動作するが、意図していない定数を使用している
```

【良い例】

```
enum ecountry {
      ENGLAND, FRANCE, …
} country;
enum eweek {
      SUNDAY, MONDAY, …
} day;
…
if ( country == ENGLAND ) {
if ( day == MONDAY ) {
if ( country == SUNDAY ) { /*  ツールでチェック可能  */←異なる定義で比較している
```

【備考】

2-1-7. マクロ定義は“ありきたりなネーミング”にしない

カテゴリ	可読性-マクロ
運用基準	推奨

【詳細説明】
ローカルなマクロ定義で、YES や NO、OK や NG などありきたりな名前をつけない。
プロジェクト全体で統一していればよいが、上記の定義を、複数箇所で行っている場合は、リバースエンジニアリングの際に誤解の原因になる。
また、モジュール化の観点からもモジュール名を伴ったネーミングにした方が良い。

【良くない例】
Ａモジュール

```
#define   OK   (1)
int     funcA ( void )
{
      return( OK ) ;
}
```

B モジュール

```
#define   OK   (0)
int   funcB ( void )
{
      return(OK) ;
}
```

【良い例】
A モジュール

```
#define   AMODULE_ERROR  (100)
int   funcA ( void )
{
      return( AMODULE_ERROR );
}
```

B モジュール

```
#define   BMODULE_ERROR  (-1)
int   funcB ( void )
{
      return( BMODULE_ERROR ) ;
}
```

【備考】

2-2. 関数

2-2-1. 関数ポインタ利用で高速化

カテゴリ	関数-ポインタ
運用基準	ノウハウ

【詳細説明】
特定の変数の値で実行する処理を分岐するような場合は、条件文を利用するのではなく、関数ポインタを配列化してアクセスすることで条件判定にかかる処理時間をなくし、高速化できる。また、可読性も向上する。

【良くない例】

```
void func ( int data )
{
	if( data	==	A ){
		operation_A();
	}
	else if( data == B ){
		operation_B();
	}
	else{
		operation_C();
	}
}
```

【良い例】

```
void (*Operation[] )(void)= { operation_A, operation_B, operation_C };

void func ( int data )
{
	if( data >= 0 ){//配列の下限チェック
		if( data < (sizeof(Operation)/sizeof(Oparation[0])) ){//配列の上限チェック
			Operation[data]();
		}
	}
}
```

【備考】
配列化する場合は、上記の【良い例】のように配列の上下限を考慮すること。

2-2-2. 無限ループに注意せよ！

カテゴリ	関数-ループ回数
運用基準	推奨

【詳細説明】
数値解析プログラムは、計算結果が収束せず無限ループにおちいる可能性がある。
ループの上限回数を決めておくなどの対策が必要。

```
#define    EPS  (0.0001)
```

【良くない例】

```
for(;; ){
        step = calcStep();
        if( fabs( step ) < EPS ){break;}
}
```

【良い例】

```
#define    LOOP_MAX      (1000)

for( i=0;    i<LOOP_MAX;    i++ ){
        step = calcStep();
        if( fabs( step ) < EPS ){break;}
}
if( i >= LOOP_MAX ){//収束しなかった
        step = EPS;
}
```

【備考】
ループ回数の上限を指定する場合は、上限ループ後の処理を考慮すること。
上記の【良い例】では、上限回数で収束しない場合は、変数 step を収束判断値としている。

2-3. コメント

2-3-1. 関数ヘッダーはわかりやすく

カテゴリ	コメント-関数ヘッダー
運用基準	遵守

【詳細説明】
関数ヘッダーの引数・戻り値説明部分には、単位、精度、範囲、意味付けを明確に記載しておくこと。
また、リエントラント関数でない場合や制限事項がある場合も明記しておくこと。

下記事例では、旧様式で記載していますが、ここでは記載形式を重要視しているのでなく、記載内容についてのガイドラインになりますので、Doxygen 形式などの記載でも問題ありません。

【良くない例】

```
/*-----------------------------------------------------------------------------
 *
 *   AcfSearchPeakLvl
 *
 *++［機　能］ 単体魚開始点から終了点までの範囲でピークレベルを探す
 *
 *++［引　数］ S_DETECTFISH   *pObj
 *                 shrot                echoID エコーＩＤ
 *
 *++［戻り値］ なし
 *
 *++［備　考］ 単体魚検出が終わった後に普通エコー（高周波と低周波別々）で呼び出すこと
 *
 *-----------------------------------------------------------------------------
 */
```

【良い例】

```
/*-----------------------------------------------------------------------------
 *
 * AcfSearchPeakLvl
 *
 *++［機　能］ 単体魚開始点から終了点までの範囲でピークレベルを探す
 *
 *++［引　数］ S_DETECTFISH *pObj     単体魚情報（魚体長[m]、深度[m]）
 *                 short            echoID エコーＩＤ（0:高周波、1:低周波、9:高低共通）
 *
 *++［戻り値］ なし
 *
 *++［備　考］ 単体魚検出が終わった後に普通エコー（高周波と低周波別々）で呼び出すこと
 *
 *-----------------------------------------------------------------------------
 */
```

【備考】

2-3-2. 流用物の変更個所を明確化

カテゴリ	コメント-不具合情報
運用基準	遵守

【詳細説明】
変更個所にはコメントを残すこと。
特に、多数のユニットで共用されるソフトウェアモジュールについては、不具合発生時に、流用元にも潜在的に存在する不具合なのか、実装を変更したために生じた不具合なのかが明確になる。
（バージョン管理に Subversion 等の構成管理ツールを利用した場合でも必要。）
変更量によっては、変更行にコメントを残す必要がない場合もある。
１） 一部分変更の場合
　変更行にコメントを残す。変更理由・動作原理について記録しておく。
２） 全体を見直し整理した場合
　変更行にコメントを残さなくてもよいが、関数ヘッダーに変更点を記載しておく。

【備考】

2-3-3. コメント追加をためらうな！

カテゴリ	コメント-コメント量
運用基準	推奨

【詳細説明】
コメント、数値マクロは、ためらわず追加すること。
（コードの意味を追いかけやすくなる。）
コメントの追加によって過去のコードとの差分が見た目上増えることを気にする必要はない。
変更時のコメント追加は、「理由」「日付」「名前」を記入することを意識する。
共有ソフトを流用したときは、以前のコメントよりも充実させること。
（コメントの陳腐化を防ぐため。）

【良い例】

```
// THS 対応時の不具合。EXT なし状態だとトーカーNULL で EXT 出力する不具合修正(2009/7/14, S.Wada)
if( size > 0 ){
	tx_talker = rx_talker; //  受信したトーカーを使う
}
else{
	tx_talker = (char*)format_talker[talker_kind]; //  メニューで選択したトーカーを使う
}
```

【備考】

2-3-4. デバイス設定はコメントに！

カテゴリ	コメント-デバイス
運用基準	ノウハウ

【詳細説明】
１）CPU のレジスタ設定
２）CPLD/FPGA の設定値
３）レジスタのアクセス定義(R/W、 Read only、 Write only、 N/A)や、ディフォルト値
４）レジスタのアクセス bit 設定(8, 16, 32)
以上に関するコメントは、ソフトウェア外部仕様書をひもとかなくて良いように、詳細に記載しておくとデバッグ上便利である。

ビットフィールド、設定の手順を明記する。→可読性が向上する。
プログラムでの設定意図について記述する。→デバッグ効率が向上する。

【良い例 1】

```
/* 外部割り込み優先順位の設定 */
sfr_outl( INTC_INTPRI,   (0 << 28) |     /* IRQ0 利用しない */
                         (0 << 24) |     /* IRQ1 利用しない */
                         (5 << 20) |     /* IRQ2 受信割り込み  */
                         (0 << 16) );    /* IRQ3 利用しない */
```

【良い例 2】

```
/* 割り込みコントロールレジスタの設定 */
sfr_outl( INTC_ICR0, 0xC0E00000 );
/*
    [30] NMI 端子がローレベルの間割り込み禁止
    [23] IRQ0-3 は独立した割り込み入力
    [22] Reserved (書き込み時常に 1 指定)
    [21] レベル入力の割り込みが CPU 受付前に取り下げられた場合、割り込み要求を保持しない
*/
```

【備考】

2-3-5. 未実装箇所を明確に！

カテゴリ	コメント-未実装
運用基準	遵守

【詳細説明】
未実装の個所にはコメントを残しておく。
後から検索しやすくするように、未実装であることを意味するキーワード（例えば“[to do]”）をプロジェクトで定義しておく。（詳細設計以降開発方針書で必ず定義し、途中参加のメンバーにも適宜周知する）
詳細設計時からのスケルトンコードレベルで未実装がわかるようにしておくと良い。
（ユースケース図等から自動ツールでソースをはき出す場合を想定。）
次版対応なども残しておくと変更すべき個所がわかって良い。

【良い例】

```
if( data == A ){
	operation_A();
}else{
	//[to do]
}
```

【備考】

2-3-6. 処理時間計測結果をコメントに！

カテゴリ	コメント-性能
運用基準	推奨

【詳細説明】
割込み禁止区間や割込み処理区間、性能に関係するタスクの処理部分などは、処理時間を計測しコメント化する。(レビューの効率化、性能要求を満たすことの確認に便利)

【良い例】

```
//    割り込み禁止(禁止時間は実測値 180μsec<2003/9/2>)
psw = vdis_psw();
uart_m38881m2( 0 ) ;            //UART 送受信処理
GetInterrptStatusM38881 (0);
vset_psw(psw);                  //割り込み許可
```

【備考】
上記事例では、ソースコード内に記載しているが、関数ヘッダーに実行環境(CPU、システムクロックなど)や機種名を記載しておく方が流用時にも役立つ。

2-3-7. ハードウェア依存の明示

カテゴリ	コメント-ハードウェア
運用基準	推奨

【詳細説明】

CPU・FPGA 依存の処理は、想定外のハードウェア（バージョン）では動作しない可能性が高い。
そのため、対応 CPU をコメントとして明示するほうが良い。
使用しているデバイス特有の処理であれば、必ず正式な型番までコメントに記載すること。
(ex.)SH-2A と書くだけでなく、SH7203 と記載

【良い例】

```
// 動作確認済みフラッシュ ROM は下記のみ
#define MAKERID          (0x04)     /* FUJITU */
#define DEVICEID_T       (0x220C)   /* MBM29DL400TC */
#define DEVICEID_B       (0x220F)   /* MBM29DL400BC */
#define DEVICEID_800B    (0x22CB)   /* MBM29DL800BA */
```

【備考】

2-3-8. 処理待ち時間の根拠を記載せよ！

カテゴリ	コメント-処理待ち
運用基準	推奨

【詳細説明】

dly_1us()や、dly_tsk()など、時間待ち関数をコールする際に、「なぜ待つのか」、「最低どれだけ待たなければいけないのか(= マージンをどの程度とっているのか)」をコメントとして記載しておくこと。

【良い例】

```
/* EEPROM の消去待ち時間.　データシートより、Max75msec */
#define　EEPROM_ERASE_TIME　(100)

void　erace_EEPROM ( void )
{
　　eeprom_write( EEPROM_ERACE_CMD );
　　dly_1msec( EEPROM_ERASE_TIME );
}
```

【備考】

2-4. 書式

2-4-1. マクロは行頭から記述する！

カテゴリ	書式-インデント
運用基準	ノウハウ

【詳細説明】マクロはかならず行の先頭から書き始めること。
マクロをスペースやタブの後に続けると、マクロを正しく処理しない開発環境がある。

【良くない例】

```
while( condition ){
    #ifdef    SH4      /* #  の前に空白→NG */
    .....
    #endif
}
```

【良くない例】（#の後のスペース、タブも NG）

```
while (condition){
#   ifdef    SH-4
  .....
#   endif
}
```

【良い例】

```
while (condition){
#ifdef    SH-4
    .....
#endif
}
```

【備考】
glibc や開発環境付属のコード(含む、ヘッダファイル)で、#の後ろにスペースが入っている場合もあるので自PJ 外から提供されるコードについては、基本的には修正不要。（開発環境により正しく処理されない時のみ修正する）

2-4-2. インデント方法の統一

カテゴリ	書式-インデント
運用基準	推奨

【詳細説明】
1）インデントにスペースを使うか、タブを使うか決めておく。
　スペースとタブが混在するのはよくない。（検索時に検索しにくくなる）
2）タブを使う場合は、タブの幅を統一する。

【良くない例】

```
/* E_ERR の行は、タブを使用し、E_OK の行はスペースが使用されている */
#define	E_ERR	(-1)
#define	E_OK	(0)
```

【良い例】

```
/*	両方の行とも、タブを使用している */
#define	E_ERR	(-1)
#define	E_OK	(0)
```

【備考】

2-4-3. 演算順序を明確にするために括弧をつける

カテゴリ	書式-演算順序
運用基準	推奨

【詳細説明】
C言語で定められている演算の優先順位にとらわれず、意識的に()を用いて演算順序を示す。

【良くない例】

```
if( rider == OK && belt == NONE && bike == NONE ){
        .....
}
```

【良い例】

```
if( (rider == OK) && (belt == NONE) && (bike == NONE) ){
        .....
}
```

【備考】
四則演算でも乗算、除算が混合する演算にも()を意識的に用いた方が、わかりやすく保守性もよい。

【事例】

```
data = x1 * x2 + y1;
```

【推奨例】

```
data = (x1 * x2) + y1;
```

2-4-4. 関数プロトタイプ宣言にも変数名を

カテゴリ	書式-関数宣言
運用基準	遵守

【詳細説明】
関数のプロトタイプ宣言にも変数名を書いておくこと。

【良くない例】

```
/* 底の累乗を求める関数 */
extern    int   exponential( int, int );
```

【良い例】

```
/* 底の累乗を求める関数 */
extern    int   exponential( int base, int x );
```

【備考】
プロトタイプ宣言は、関数本体部分を複写することで戻り値や引数の型宣言間違いなどを抑止できる。
但し、関数名の後ろのスペースは削除すること、（2-4-5）

2-4-5. 関数名の後にスペースを

カテゴリ	書式-関数表現
運用基準	ノウハウ

【詳細説明】
関数定義時の関数名の後（括弧の前）にスペースを入れ、その関数を参照する箇所(含む、外部参照宣言)ではスペースを入れない区別をつける。これにより、Grep 検索の結果表示が、関数定義かそれを参照する箇所のいずれかに限定され、検出しやすくなる。

【良い例】

```
void      sub  ( int a )        //スペースを入れる
{
      .....
}


extern    void   sub( int );    //スペースなし
void      abc  ( void )         //スペースを入れる
{
      .....
      sub( 5 );                 //スペースなし
      .....
}
```

【備考】

2-4-6. 半角カナは使わない

カテゴリ	書式-コメント
運用基準	推奨

【詳細説明】
半角カナは、文字化けする可能性が高いため、使用しないことを推奨する。（開発環境によっては、コンパイルエラーになる場合がある。）
逆に、英数字、スペースの挿入に関しては半角を推奨する。

【良くない例】

```
a--;    /* ﾃﾞｸﾘﾒﾝﾄ */
```

【良い例】

```
a--;    /* デクリメント */
```

【備考】
開発環境によっては、2byte コード(例、S-JIS)を使用するとコンパイルエラーになる場合があります。

2-4-7. 中括弧は省略しない

カテゴリ	書式-中括弧
運用基準	遵守

【詳細説明】
if 文や while 文を使用する際、その後の処理が 1 行、または 1 文の時にも中括弧を省略しないこと。
省略するとプログラム作成者の意図が不明瞭になる。また、コードを追加する際に不具合の原因になる等の弊害がある。（if 文、while 文が入れ子構造になっている際に問題が発生しやすい。）

＜1 行の例＞
【良くない例】
```
if( condition )    return;
```

【良い例】
```
if( condition ){
    return;
}
```

＜1 文の例＞
【良くない例】
```
if( condition )
    while( condition )
        n++;
```

【良い例】
```
if( condition ){
    while( condition ){
        n++;
    }
}
```

【備考】下記のようなプログラムで、if 文が真の時の部分にプグラムを追加する時に{}を付け忘れると意図通りのプログラムにならない。（下記例では、関数 func()を追加）

〈修正前〉
```
if( condition )
    while( condition )
        n++;
```

〈修正後（不具合あり、中括弧の記述忘れ）〉
```
if( condition )
    func();                 ←追加コード
    while( condition )      ←左記コードが condition の状態によらず必ず実行される
        n++;
```

〈正しい修正〉
```
if( condition ){
    func();
    while( condition ){
        n++;
    }
}
```

2-4-8. マクロは括弧で括る

カテゴリ	書式-マクロ
運用基準	遵守

【詳細説明】
定数を表すマクロは、必ず括弧で括ること。
（マクロ影響範囲の誤りを防ぐため。）

j = (100+200) * i = 300 * i;
を計算したい。

【良くない例】

```
#define    TOTAL     100+200

   j = TOTAL * i;
```

は、

```
   j = 100+200 * i
```

と解釈され、200*i が先に演算されてしまう。

【良い例】

```
#define    TOTAL     (100+200)

   j = TOTAL * i;
```

は、

```
   j = (100+200) * i
```

と解釈され、(100+200)が先に演算され、その結果と i が乗算される。

【備考】

2-4-9. 無限ループの記述方法

カテゴリ	書式-無限ループ
運用基準	推奨

【詳細説明】
無限ループの記述方法には、主に for(;;)、 while(1)、 do~while(1)の 3 種類あるが、
for(;;)に統一するのが良い。
while(1)、 do~while(1)は、条件式記入忘れと区別がつけにくい。

【良くない例】

```
while( 1 ){
    .....
}
do{
    .....
}while( 1 );
```

【良い例】

```
for(;;){
    .....
}
```

【備考】

2-4-10. 命名規則を統一

カテゴリ	書式-命名規則
運用基準	推奨

【詳細説明】
変数、関数、定数などに命名規則を設けておくことで、可読性と保守性が向上する。
少なくとも、プロジェクト内で統一をとること。
例えば、以下のようなルールをプロジェクト内で決めておく。
１）変数は小文字で始める。ローカル変数の最初には、'_'を付加する
２）関数は大文字で始める
３）定数はすべて大文字
４）キャメルケース/アンダーバー区切り
５）ハンガリアン記法を使用するかどうか

【良くない例】

```
#define   E_ERR            (-1)
#define   E_OK             (0)
#define   tdma_wait_timer  (120000)

int   isEndTDMAmonitor ( void )
{
      static int  waitTimer = tdma_wait_timer;
      static int  SIZE = 0;
      int         ret=E_ERR;

      setTimerMs( &waitTimer );
      if( waitTimer > 0 ){
            SIZE++;
            ret = E_OK;
      }
      return( ret );
}
```

【良い例】

```
#define   E_ERR            (-1)
#define   E_OK             (0)
#define   TDMA_WAIT_TIMER  (120000)

int   isEndTDMAmonitor ( void )
{
      static int  _waitTimer = TDMA_WAIT_TIMER;
      static int  _size = 0;
      int         ret=E_ERR;

      setTimerMs( &_waitTimer );
      if( _waitTimer > 0 ){
            _size++;
            ret = E_OK;
      }
      return( ret );
}
```

【備考】

2-4-11. 繰り返し処理の継続条件を条件部に明示

カテゴリ	書式-ループ処理
運用基準	推奨

【詳細説明】
for 文、while 文の継続条件を、コードブロックに記載するのではなく、条件部に記載する。
条件部を積極的に利用して、継続条件をはっきりさせること。

【良くない例】

```
while( 1 ){
	process();	// process
	if( ending() ){
		break;
	}	//終了条件
}
```

【良い例】

```
do{
	process();	// process
}while( !ending() );
```

【備考】

2-4-12. long 値を表すには大文字の ’L’ を使う

カテゴリ	書式
運用基準	推奨

【詳細説明】
小文字の 'l' は数字の '1' と混同されやすいので、大文字の 'L'を使うのがよい。
これが特に紛らわしいのは、整数表記が long を意味する場合である。

【良くない例】

```
long      data=0l;
```

【良い例】

```
long      data=0L;
```

【備考】
変数名に小文字の’l’を使用した時にも同様の紛らわしさがある。下記は、一例です。
6 行目が数字のいちで 7 行目が小文字のエルです。

2-4-13. 中括弧はネストの始点と終点で揃える

カテゴリ	書式-インデント
運用基準	推奨

【詳細説明】
中括弧を閉じる位置は、【事例 B】のように、ネストの始点と終点が揃うように配置することを推奨する。

【例】
下記【事例 B】の方が可読性がよい

【事例 A】
```
while (condition) {
    .....
    }
```

【事例 B】
```
while (condition) {
    .....
}
```

【備考】
流用ソースファイルを修正する時は、流用元のルールに従う方が良い。本項目の推奨に従って修正し複数種類の方法を混合させないこと。

2-5. 処理系

2-5-1. 処理系依存表現の禁止

カテゴリ	処理系
運用基準	遵守

【詳細説明】
pragma 文や long long など特定の処理系でしか使えない型の利用は極力さける。
特にライブラリなど共通性をもつプログラムでは使わないこと。
（アプリレベルではそれが非常に有用な場合でも、ヘッダーを利用するなどしてソースコードとしての一般性を失わない工夫をすること。）

【良くない例】
long long 型

【備考】

2-5-2. 負数に対する割り算に注意

カテゴリ	処理系
運用基準	注意点

【詳細説明】
負数に対する割り算は、処理系によって結果が異なる場合がある。

【事例】

-1/10
① 0　余り　　-1
②-1　余り　　9

①のケースが多数。

【備考】
下記は、C99(JISX3010(X3010：2003（ISO/IEC 9899：1999）))の「6.5.5 情除演算子」からの抜粋でC99では、①②どちらも規定にあった結果となっている。

a = -1, b = 10, (a/b) = 0 or -1, (a%b) = -1 or 9
① 0 * 10 + (-1) = -1
② -1 * 10 + 9 = -1

整数同士の除算の場合、/演算子の結果は、代数的な商から小数部を切り捨てた値とする。商a/bが表現できる場合、式(a/b)*b+a%bは、aに等しくなければならない。

【参考】
Excel 2010でMOD関数を使用すると②の結果となる。

2-5-3. volatile 修飾された変数が間違ってコンパイルされることに注意	
カテゴリ	処理系
運用基準	注意点
【詳細説明】 言語規格に従った volatile 動作が必要な場合に、自分のコンパイラがどのように動作するか知っておく必要がある。	
【備考】	

2-6. 条件文

2-6-1. 範囲チェックは最大/最小の両方で

カテゴリ	条件文
運用基準	推奨

【詳細説明】
明示的に最小値・最大値との範囲チェックを行っておけば、不具合は発生しにくくなる。
（仕様を明確に表現するという意味でも大事）

```
#define    DATA_MAX     (100)
#define    DATA_MIN     (-10)
```

【良くない例】

```
if ( data <= DATA_MAX ){       // 最小側のチェックを行っていない
        process( data );
}
```

【良い例】

```
if( (data >= DATA_MIN) && (data <= DATA_MAX ) ){
        process( data );
}
```

【備考】
変数を配列番号に使用する場合も、範囲チェックを行い負数側のチェックも行うこと。

2-6-2. 比較対象の明示

カテゴリ	条件文
運用基準	推奨

【詳細説明】
C言語での論理式条件判定の内部表現は、
　真 ： 非0 、 偽 ： 0 だが、
変数の値だけで条件判定を行うのではなく必ず判定対象となる値との一致を条件とする。

【良くない例】

```
if( data ){        // 0 であるべきか、非 0 であるべきか不明
    process();
}
```

【良い例】

```
if( data != 0 ){  //   条件値を明示し仕様を明確化
    process();
}
```

【備考】

2-6-3. 条件文の中で、代入式を用いない！

カテゴリ	条件文
運用基準	遵守

【詳細説明】
条件文の中で、代入式("=")を用いてはいけない。
条件文の中に"="を用いると、代入式の"="なのか比較式の"=="の誤記なのかの判断がつかない場合がある。
また、デバッグ時に変数の代入と比較が同時に行われてしまうのでデバッグがやりにくくなる。

【良くない例】

```
if( n = func() ){        //”n == func()”と記述すべきなのかの区別がしにくい
    .....
}
```

【良い例】

```
n = func();
if( n ){
    .....
}
```

【備考】

2-6-4. switch 文の default のスペルミスに注意

カテゴリ	条件文-switch 文
運用基準	遵守

【詳細説明】

switch 文内に存在する"default"のスペルミスに注意する。この default を間違えて記入しても、switch 文はラベルと判断するため、ビルドエラーにならない可能性がある。

下記いずれかの方法により default スペルミスを防止すること

1）　静的解析ツールにより検出する(PGRelief の a 指摘 pgr0009 で検出可能)

2）　全パステスト、または単体テスト時に default のパスに正しく通ることを確認する

3）　ソースコードレビュー時に必ずスペルを確認する

また、編集用ソフトウェアで指定した文字列の色を変更できる機能があれば、その機能を使用して C 言語用定義文字に色を付ければ、誤字を発見しやすくすることが可能になる。（参考欄参照）

【良くない例】

```
switch (x) {
    case 1:
            printf("1¥n");
            break;
    defautl:                    // default が正しい
            printf("other¥n");
            break;
}
```

defautl はラベルと判断されるので、ビルドエラーにならない。ただし、Warning レベルによっては、検出できる可能性もある

【備考】

・編集用ツールで予約語などの特定のワードを色で区別可能

【参考】Mifes を使用した時の事例

```
1157
1158    switch(id){
1159    case DB_ID_RTE_GPS1:
1160        sel = PRI_GPS1;
1161        break;
1162    case DB_ID_RTE_GPS2:
1163        sel = PRI_GPS2;
1164        break;
1165    case DB_ID_RTE_LAN:
1166        sel = PRI_LAN;
1167        break;
1168    default:
1169        break;
1170    }
1171    if(sel < 0){
1172        return;
1173    }
```

2-6-5. if-else if 文は、１種類の変数に注目して表現すること

カテゴリ	条件文
運用基準	推奨

【詳細説明】
if-else if 文は、１種類の変数に注目して表現するのがよい。
この形式で何種類もの変数を使うと、論理が追いにくくなる場合がある。
（特に、条件が各々独立している場合。）

【良くない例】

```
if( data == 10 ){
	funcA();
}
else if( size == 100 ){
	funcB();
}
else{
	funcC();
}
```

【改善例】

```
if( data == 10 ){
	funcA();
}
else{
	if( size == 100 ){
		funcB();
	}
	else{
		funcC();
	}
}
```

【備考】

2-6-6. 比較対象の数値は前置きにする

カテゴリ	条件文
運用基準	ノウハウ

【詳細説明】
if 文の比較対象数値を前置きにすると、間違いをコンパイルで検出できる。

【例】この記述により、if(1 = data)という間違いをした場合にコンパイルエラーで検出できる

```
if( 1 == data ){
}
```

【備考】

2-6-7. 条件文内の変数インクリメントに注意

カテゴリ	条件文
運用基準	注意点

【詳細説明】
条件文に変数のインクリメントが含まれている場合、例題のように、インクリメントが行われない場合があるので、このような記述はさけること。

【良くない例】

```
if( (n++ == 10) && (m++ == 10) ){
    .....
}
```

一つ目の条件が FALSE になった場合に、２つ目の条件判断が実行されず、m++が実行されない。

【改善例】n、m の両変数が必ずインクリメントされる必要がある場合は、事前に各変数の内容をインクリメントしてから比較を行う。

```
n++;
m++;
if( (n== 10) && (m == 10) ){
    .....
    }
}
```

【備考】
変数の演算だけでなく、関数の戻り値を比較対象にした場合も同様のことになるので注意すること。
下記事例では、funcA の戻り値が FALSE の場合に funcB()が実行されず、mに値が設定されない場合がある。

【良くない例】

```
if( (n=funcA()) && (m=funcB() ){
    .....
}
```

【改善例】

```
n = funcA();
m = funcB();
if( n && m ){
    .....
}
```

2-7. 変数

2-7-1. union の取扱い-エンディアンフリーな表記方法

カテゴリ	変数-共用体
運用基準	推奨

【詳細説明】
union を利用する場合、endian の違いから、CPU が異なると互換性がなくなる可能性がある為、注意が必要である。
union 使用時は、具体例にあるように endian を自動判別するような仕組みを利用することを推奨する。

【良い例】　　endian を自動判定する場合の例。
16bit から 8bit を取り出す場合に、エンディアンに依存しないで LSB、MSB を取り出す方法を示している。
LSB は、u.c[LSB] で取り出せる。

```
//* Big Endian : MSB=0, LSB=1,    Little Endian : MSB=1, LSB=0

static const unsigned short _endianIndex = 0x0001;

#define    MSB        (*((unsigned char*)&_endianIndex+0))
#define    LSB        (*((unsigned char*)&_endianIndex+1))

union{
        unsigned short        w;
        unsigned char         c[2];
} u;

        u.c[MSB] = *str++;
        u.c[LSB] = *str++;
```

【備考】

2-7-2. 戻り値用変数名も値の意味が連想される名前にせよ

カテゴリ	変数-名称
運用基準	推奨

【詳細説明】

変数名として ret や ans などを使う例と、buffSize 等、値の意味が連想されるような表現を使う例がある。
メリット、デメリットがそれぞれにあるので、うまく使い分けること。

ret、ans：　メリット：戻り値に利用されていることがわかりやすい。
　　デメリット：物理的意味がわからない。

buffSize　：　メリット：物理的意味がわかりやすい。
　　デメリット：戻り値に利用されていることがわかりづらい。

【良くない例】

戻り値に変数値と固定値が混ざっていると、ret の意味がわかりにくくなる。
-1 が何を示しているのかわからない。
関数ヘッダーや、ret = -1 と記述されている箇所にコメントを追記するだけでも理解しやすくなる。

```
if( condition ){
    ret =size;
}else{
    ret = -1;
}
return( ret );
```

【備考】

2-7-3. オート変数の使い回しは、しない！

カテゴリ	変数-名称
運用基準	遵守

【詳細説明】
変数は「なんとなく使用」してはならない。
自分で厳密に「定義」し、その「定義どおりに使う」こと。
（一つの変数をある時はフラグに使い、またある時は演算結果の保存に使ったりすると可読性を悪くし、不具合の原因になる。）

【良くない例】
変数 qf が使い回されている。

```
void func ( int mode, int data )
{
	int qf;

	qf = GetSensorFlag( mode );
	if( qf == SENSOR_OK ){
		SetSensor( data );
	}else{
		SetErrorFlag( qf );
	}

	for( qf=0; qf<MAX_PORT; qf++ ){
		ResetPort( qf );
	}
}
```

【備考】
変数宣言の箇所には、下記のような情報を記載しておくと使いまわしをしにくくなるとともに、コードを理解することも容易になる。
- 値の意味
- 値のとりうる範囲
- 精度
- 単位

2-7-4. 変数名は、定義どおりに

カテゴリ	変数-名称
運用基準	推奨

【詳細説明】
変数名は、定義にしたがって名前づけすること。
論理、物理量を表現していることがのぞましい。
（フラグなのか、添え字なのか、数量をあらわすのか、整数なのか、実数なのか、ポインタなのか、
正論理（ＮＺ）なのか否定論理（０）なのか、などを彷彿させる。）

【事例】

fComplete：	完了フラグ
speed：	速度（整数）
xSpeed：	速度（実数）
*pSpeed：	速度（ポインタ）
sum：	和
err：	NZ でエラー、0 で OK

【備考】

2-7-5. 名称は仕様書から取ろう

カテゴリ	変数-名称
運用基準	推奨

【詳細説明】
仕様書に定義されている情報名は、仕様書の記載にあわせて名称付けするほうが良い。
仕様書との対応が明確になりやすい。
ただし、仕様書で情報名が適切に選択されていることが条件である。
また、
１） 仕様書での表記がＣ表記上使用できないもの
２） 仕様書での表記が変数名規約と合致しないもの
については、仕様書記載通りの名称付けはできないが、なるべく近い表現にすること。

【備考】

2-7-6. 予約語まがいの名前を変数名に使わない

カテゴリ	変数-名称
運用基準	遵守

【詳細説明】
可読性の低下につながるので、変数名を予約語に近いものにしてはいけない。

【良くない例】（予約語を先頭に使い、意味不明の文字を付加している場合）

```
default;
auto1;
short2;
signedx;
```

【良い例】（予約語を先頭に使っているが、意味のはっきりした語を付加している場合

```
defaultSize;
autoRange;
```

【備考】

2-7-7. オート変数名は単純に

カテゴリ	変数-名称
運用基準	推奨

【詳細説明】
物理量をもたない自動変数は、単純な変数名にする。

【良い例】
```
int   i, j, k, m, n;
```

【備考】
i と j は、見間違えやすいので、二重ループで両方を使うことは避けたほうがよい。

```
int i, j;
for( i=0; i<100; i++ ){
    for( j=0; j<10; j++ ){
        .....
    }
}
```

2-7-8. 変数名からその目的が分かるように

カテゴリ	変数-名称
運用基準	推奨

【詳細説明】
変数名は、その名前から何をする為のものであるかを判断出来るように決めるべき。
また、正・負論理も名前判別出来るのが望ましい。
複数の単語を組み合わせて、意味をわかりやすくすること。

【良くない例】

```
static int  _batteryVolt;        //  電池電圧エラー  0:OK, 1:NG
static int  mode;                //表示モード値
```

【良い例】

```
static int  _batteryVoltError;  //  電池電圧エラー  0:OK, 1:NG
static int  _modeDisplay;       //表示モード値
```

【備考】

2-7-9. ビット反転、ビットシフト時の汎整数拡張

カテゴリ	変数-演算規則
運用基準	注意点

【詳細説明】
ビット反転、ビットシフトをした場合、汎整数拡張が行われることを認識しておくこと。
（汎整数拡張：int 未満の整数型が int/unsigned int に自動変換されること。）

int 未満の型の場合、int 型の変数で演算する、あるいは演算後明示的にキャストするのが望ましい。

【良くない例】

```
unsigned char        a, b;

b = a << 4;
```

と記述した場合、a のシフトは一般的に unsigned int で行われる為、コンパイラによっては、次の warning を出す可能性がある。「右辺のサイズの方が大きいため、正しく代入されない可能性が有り。(左 ： unsigned char、 右 ：int) 」

対策としては変数を unsigned int に置き換えて演算させるか、【良い例】のようにキャストする。

【良い例】

```
unsigned char        a, b;

b = (unsigned char)(a<<4);
```

【備考】
数量データに対して、ビット反転、ビットシフトを行うのは避けること。
汎整数拡張を意識しなければならないことに加えて、可読性が悪くなる。

2-7-10. ビットシフトの仕様はコンパイラ依存

カテゴリ	変数-演算規則
運用基準	注意点

【詳細説明】

ビット右シフトの仕様は一般的に signed なら算術シフト、unsigned なら論理シフトとなっている。

```
long            s_data = 0xFFFFFFFF;
unsigned long   u_data = 0xFFFFFFFF;

s_data >>= 8;   /* 結果 = 0xFFFFFFFF *//* 符号拡張されながら右シフトされる */
u_data >>= 8;   /* 結果 = 0x00FFFFFF */
```

しかし、C言語の規格上、明確に定義されていないことを知識として持っておくこと。（対策としては具体例を参照）

【良い例】

コンパイラの仕様が一般的な仕様（signed：算術シフト、unsigned：論理シフト）と異なるかどうかを判断する為に、下記をソースコードに追記して確認すること

```
/*
 * 一般的に
 *      符号付き右シフトは算術シフト（符合で埋める）
 *      符号無し右シフトは論理シフト（０で埋める）
 * となっているが、Ｃの規格上は定義されていない為、本ファイルにてコンパイラの仕様を確認する。
 * 上記仕様でないコンパイラの場合、本ファイルでコンパイルエラーが発生する。
 * （配列の添え字に 0 を使用してエラーを発生させている）
 * int が３２ビットでない場合もある為、char 型で判断するように変更。
 * (signed char)0xff とすると vc++ では、
 * 「warning C4310: キャストによって定数値が切り捨てられました。」
 * が表示される為、(signed char)-1 とした。
 */

/* 符号付きの右シフトが算術シフトかどうか */
#define   SHIFT_SIGNED_USES_SAL    ((unsigned char)((signed char)-1 >> 1) == 0xFF)

/* 符号無しの右シフトが論理シフトかどうか */
#define   SHIFT_UNSIGNED_USES_SHL      (((unsigned char)0xFF >> 1) == 0x7F)

char      __assertion_shift_spec_signed_        [SHIFT_SIGNED_USES_SAL];
char      __assertion_shift_spec_unsigned       _[SHIFT_UNSIGNED_USES_SHL];
```

参考：（一般的な仕様の場合の結果例）

```
signed char     cx = -1;
signed char     cy;

cy = ((unsigned char)cx >> 2);      /* cy は、0x3f となる */
```

【備考】

算術シフト(shift arithmetic／符号充填シフト)：シフトによって空いたビット部分と符号ビットを同じもので詰めるシフト。

論理シフト(shift logical／0 充填シフト)：シフトによって空いたビット部分を 0 で詰めるシフト。

補足：PGRelief - 富士通の見解

コンパイラに依存するのは、signed の場合のみで、unsigned なら必ず論理シフト（０充填）であるとのこと。

2-7-11. 符号有変数と符号無変数を混在させて比較しない

カテゴリ	変数-演算規則
運用基準	遵守

【詳細説明】
符号有/無のどちらで比較されるのかは、それぞれの型に依存する規格となっている為、比較すべき型にキャストして比較するようにする。

補足：　ANSI C、およびC++の場合の定義
long double、double、float、unsigned long、long、unsigned int、int の順に型を調べ、最初に現れた型に両式とも変換。
但し、long サイズ >= int のサイズで long int と unsigned int の組み合わせの場合、 unsigned long int 型に両式とも変換。

【良い例】(int が 32bit の場合)

```
short      short_minus_1 = -1;

if( short_minus_1 < (unsigned short)0xFFF0 )      // 条件により符合有で判定：結果 真
      (-1 と 65520(=0xFFF0)の比較となる)

if( short_minus_1 < (unsigned int)0xFFFFFFF0 )    // 条件により符号無で判定：結果 偽
      (4294967295(=0xFFFFFFFF)と 4294967280(=0xFFFFFFF0))との比較となる

if( short_minus_1 < (unsigned long)0xFFFFFFF0 )  // 条件により符号無で判定：結果 偽
      (4294967295(=0xFFFFFFFF) と 4294967280(=0xFFFFFFF0))との比較となる
```

【備考】
両方共、int 以下（unsigned int を除く）の場合、signed int で比較
上記以外の場合、signed と unsigned の比較では原則 unsigned で比較される

2-7-12. 符号有から無のキャストは変数と同じbit長で

カテゴリ	変数-演算規則
運用基準	注意点

【詳細説明】
符号有から無のキャストの場合、その変数と同じbit長でキャストしないと意図に反する可能性がある。
詳細は具体例を参照。

【良い例】

```
short           signSHORT = -1;
unsigned long   unsignLONG;

unsignLONG = (unsigned long)signSHORT;
// unsignLONG = 0x0000FFFF ではなく 0xFFFFFFFF になる。

unsignLONG = (unsigned short)signSHORT;
// unsignLONG = 0x0000FFFF となる。
// (unsigned short)signSHORT でキャストした時点で符号なしとなり 65536 と扱われ、この値を符号なしの long としてキャストされるため 0x0000FFFF となる。
```

【備考】

2-7-13. 比較時の汎整数拡張に注意！

カテゴリ	変数-演算規則
運用基準	推奨

【詳細説明】
int 型は、処理系によって扱われる bit 数が異なる（16bit、32bit、64bit）。
比較の場合も汎整数拡張が発生する場合がある為、次のようなケースは注意すること。

```
short       n = 0xFFFF;
if( n == 0xFFFF )
```

int が 32 bit の場合、真になることはない。
n は汎整数拡張され 0xFFFFFFFF となり、 0x0000FFFF との比較になる為。
例えば if(n == (short)0xFFFF) 等と適切なキャストが必要となる。

※静的解析ツール(PGRelief)の pgr0001(a 指摘)で検出可能

上記以外の例。

```
signed char     c = -1;
```

【良くない例】

```
if( c == 0xFF)    //…   真になることはない。
// int 型で比較され、-1 と 0xFF(=256)の比較となってしまう為。
```

【良い例】比較元変数 c の型にキャストしてから比較する。

```
if(c == (signed char)0xFF )    //… 真になり得る。
```

【備考】

2-7-14. 全角文字列を扱う場合の比較に注意

カテゴリ	変数-演算規則
運用基準	遵守

【詳細説明】
全角コードが混在する文字列を扱うケースでは、0x81 等のコードが含まれる。
char 型変数が符合有りの場合、c = 0x81 で以下の処理が呼び出されると真の処理が行われてしまう。

```
void function ( char c )
{
    if( c < ' ' ){      // 空白コード（0x20）より小さいか？
        .....
    }
    .....
}
```

全角を含む文字比較処理がある場合、unsigned char にキャストするか、unsigned char のバッファで処理する必要がある。

【良くない例】

```
void function ( char c )
{
    if( c < ' ' ){      // 空白コード（0x20）より小さいか？
        procedure();
    }
}
```

【良い例】

```
void function ( char c )
{
    if( (unsigned char)c < ' ' ){      // 空白コード（0x20）より小さいか？
        procedure();
    }
}
```

【備考】

2-7-15. 負値をシフトで割算しても期待値にならない

カテゴリ	変数-演算規則
運用基準	注意点

【詳細説明】
数値を 1/2 にする際、右ビットシフトを利用する場合があるが、次のような場合は、期待値にならない場合がある為、注意。
n = -1 >> 1;
上記、結果は何回実行しても -1 にしかならない。
負の値を取るケースでは、通常の除算演算子を使用すること。
やむおえずシフトを使用したい時は、備考欄を参照。

```
int n;
n = -1;
```

【良くない例】

```
n = n >> 1;      /* 1/2 にはならず、-1 のまま */
```

【良い例】

```
n = n / 2;
```

【備考】
低性能 CPU を使用する場合で、シフトで高速化を実現したい場合は下記のような対応で実現可能

```
if( n == -1 ){
    n = 0;      /* -1/2=0 */
}
else{
    n = n >> 1;
}
```

2-7-16. int 型は 16 ビット符号付整数の範囲でのみ使用せよ

カテゴリ	変数-宣言
運用基準	遵守

【詳細説明】

int 型のサイズは ANSI の定義では以下のようにしか定められておらず、CPU のアーキテクチャやコンパイラに依存するためむやみに使用すると互換性(再利用) の時に問題となる。

sizeof(short) ≦ sizeof(int) ≦ sizeof(long)

int 型の使用は避けるべきであるが、その CPU における最も効率の良いコードが出力される可能性があり、その点を意識する場合には、short 型で収まる使用目的に限定し採用する。

【良くない例】

```
int   i;

// 条件が 16bit を超えて int が 16 ビットの場合は流用できなくなる
for(i=0; i<0x00200000; i++){
}
```

【備考】

2-7-17. char 型には signed、 unsigned をつけよ

カテゴリ	変数-宣言
運用基準	遵守

【詳細説明】

1byte の比較、算術演算を行う場合には、char 型が符号付きなのか、符号無しなのかは処理系依存であるため、signed、unsigned は必ずつけること。

【良くない例】

```
char        m, n;

<char 型が signed char の場合>
    m = 0xFF;       //mが－１と扱われる
    if( m < 0 ){
        .....       //こちらが実行されます。
    }
    else{           //m が正の場合
        .....
    }

<char 型が unsigned char の場合>
    m = 0xFF;       //mが 255 と扱われる
    if( m < 0 ){
        .....       /m の値がどんな値でも実行されることはない。
    }
    else{
        .....       //こちらが実行されます。
    }
```

【良い例】

```
signed char     m;      //値の範囲が、-128～127
unsigned char   n;      //値の範囲が、0～255
```

【備考】

2-7-18. 変数宣言と初期値代入について

カテゴリ	変数-宣言
運用基準	推奨

【詳細説明】
変数の宣言と初期化を同時に記載してもよいが、未初期化の変数宣言と同じ行に記載しない。
初期化の有無が判断しにくく、可読性を損なう。

【良くない例】
```
int  size, ret = 1;
```

【良い例】
```
int  size;
int  ret=1;
```

【備考】

2-7-19. typedef で見やすく

カテゴリ	変数-宣言
運用基準	推奨

【詳細説明】
構造体は typedef で型定義したほうが見やすい。

【良い例】

```
typedef struct {
    short    echoID;      /* 設定対象のエコー指定 （ECHOID_A、ECHOID_B） */
    short    level;       /* 海底レベル値（Max 255 digit） */
} ESPUT_BOTTOMLVL;
```

【備考】

2-7-20. AUTO変数のサイズはスタックを意識しろ

カテゴリ	変数-AUTO変数
運用基準	注意点

【詳細説明】
AUTO 変数で配列、構造体などを宣言する場合、スタックサイズをオーバーする可能性があるので十分に注意して宣言すること

【良くない例】

```
void        func (void)
{
double    data[256];    // 2048(=8*256)byte のスタック領域が使用される

.....
}
```

【備考】
PC 上で動作させることを目的で実装されたコードは、スタックサイズを意識していない可能性があるので、流用する場合は、要注意。

2-7-21. 非負物理量であっても、符号付き変数として宣言しよう

カテゴリ	変数
運用基準	ノウハウ

【詳細説明】
0 と正値しかとらない物理量であっても、符号付き変数として宣言しよう。
下記事例のような場合に異常判定を行うことが出来ます。

```
//ヘッダー部 4 文字、データ部 6 文字のフォーマット
//通信データとして送られ送信側機器の不具合でフォーマットに不具合が有る場合
char title1[] = { “$123FURUNO” };        //正常なデータ(ヘッダー4 文字($123)とデータ 6 文字(FURUNO))
char title2[] = { “$12” };                //異常なデータ(ヘッダーが不足、データもなし)
```

【対策前】

```
void func ( void )
{
    unsigned int    dsize;

    dsize = strlen( &title2[0] ) – 4; //(-4)：ヘッダー分を引く
                                      //dsize が符号なし変数なので-1 にはならない。
    if( dsize > = 6 ){        //ヘッダー部分を除いたサイズが 6 文字以上なら正常と見なしたい
        //unsigned int dsize なら正常と見なされる。
        .....
    }
    else{
        .....
    }
}
```

【対策後】

```
void func ( void )
{
    int    dsize;

    dsize = strlen( &title2[0] ) – 4; //(-4)：ヘッダー分を引く
                                      //dsize が符号あり変数なので-1 となる。
    if( dsize > = 6 ){        //ヘッダー部分を除いたサイズが 6 文字以上なら正常と見なしたい
        .....
    }
    else{
        //int dsize なら異常と見なされる。
        .....
    }
```

【備考】

2-8. 引数

2-8-1. 構造体はアドレス渡しで効率化

カテゴリ	引数-構造体引数
運用基準	推奨

【詳細説明】
サイズの大きな構造体を引数に取る場合、実体渡しよりもアドレス渡しを行ったほうが処理効率が上がる。
（スタックの使用量が減り、引数を受け渡す時のコピー回数削減で効率が上がる。）
但し、直接アクセス可能となる危険性があるので、参照のみ行う場合は const 宣言を行う。

```
typedef struct{
	int		total;
	char		name[MEMBER_MAX][NAME_MAX];
}MENBER;
MENBER		list;

【良くない例】
int	IsMember ( char *target, MENBER list )
{
	.....
}
void func ( void )
{
	if( IsMember( "taro", list ) ){	/*  構造体の実体渡し  */
		.....
	}
}

【良い例】
int	IsMember ( char *target, const MENBER *list )
{
	.....
}
void func ( void )
{
	if( IsMember( "taro", &list ){	/*  アドレス渡し  */
		.....
	}
}
```

【備考】

2-8-2. 入力引数の const は積極的に宣言しよう

カテゴリ	変数-宣言
運用基準	推奨

【詳細説明】

void PutString(char *pText) // 文字列を出力
のような関数で、pText を読み出しにしか利用していない場合は、
void PutString(const char *pText) と、積極的に const 宣言をした方が望ましい。
これにより、読み出し専用ならば const 宣言を付けることで、プログラム内で書き込みを行う間違った実装をした場合にコンパイラや静的解析で事前に発見できるメリットもある。

【良くない例】

```
static const char     STR_hello[] = { "HELLO" };

void PutString ( char *pText )
{
    .....
}

void func ( void )
{
    PutString( &STR_hello[0] ); ←compiler によっては warning を出力する場合が有る。
}
```

【備考】

本対応により、関数は、const char *、char * 両方の型を受けることが可能で、
もし、const 宣言がない場合、原則 char * しか受けることが出来ない。（コンパイラによっては warning を出力する場合有り）

2-8-3. 関数の引数(またはサイズ)を多くしないこと

カテゴリ	引数ーサイズ
運用基準	注意点

【詳細説明】
関数引数の個数(またはサイズ)について、多くなりすぎないように注意すべきである。

【良くない例】

```
double    calc ( ind x, int y, int z, int d1, int d2, int d3, int year, int month, int day )
```

【良い例】

```
typedef struct{
    int   x;          /* x 座標(m) */
    int   y;          /* y 座標(m) */
    int   z;          /* z 座標(m) */
    int   d1;         /*  定数 1 */
    int   d2;         /*  定数 2 */
    int   d3;         /*  定数 3 */
    int   year;       /*  西暦(1980-) */
    int   month;      /*  月(1-12) */
    int   day;        /*  日(1-31) */
}TYPE_CALC;
double      calc ( TYPE_CALC *pt )
```

【備考】
引数は一般的には、4 個から 6 個以内とし、それ以上になる場合は構造体などを利用する。
引数がレジスタで引き渡される数が 4 個から 6 個でそれ以上の場合は、スタックで引き渡され効率が悪くなるため。

2-9. 信頼性

2-9-1. 関数の戻り値チェック、エラー処理を忘れずに！

カテゴリ	信頼性-エラー処理
運用基準	遵守

【詳細説明】
エラー発生を戻り値で知らせる関数を使用する場合、呼び出し元で適切なエラー処理を行うこと。
（fopen 関数使用時など。）

【良くない例 1】

```
fp = fopen( "file_name", "w" );
c = fgetc( fp );
```

【良くない例 2】

```
if( ( fp = fopen("file_name", "w") ) == NULL ){
    /* エラー処理 */
}
c = fgetc( fp ); // 上記でエラーの場合もこの処理が実行される
```

【良い例】

```
fp = fopen( "file_name", "w" );
if( fp == NULL ) {
    /* エラー処理 */
}
else{/* fopen が正常リターン時のみ実行する */
    c = fgetc(fp);
    .....
}
```

【備考】
機械的に戻り値だけをチェックすると【良くない例 2】のようになるので、エラー処理後に通常動作の箇所が実行されないようにすること。
また、単純なエラー戻り値だけでなく、例えば取得バッファ不足などの処理も考慮して実装すること。100byte を取得したいのに、100byte 取得できなかった時とか、10 文字を出力したいのに 8 文字しか出力できなかった時など。

2-9-2. 初期化関数のコール回数に注意！

カテゴリ	信頼性-初期化
運用基準	遵守

【詳細説明】
デバイスの初期化関数は、ハードの電源が入ってから 1 回だけ初期化すべきものについては、2 回以上実行しないこと。
(ex: バスコントローラの初期化、キャッシュの設定、など)

特に、Boot からアプリケーションにジャンプした後の再初期化処理など。
(ソフト的な初期化とハード的な初期化を分けるべき)

【備考】
関数内で２回目以降の呼び出しを検知し、何もせずにリターンする工夫で対応する事も可能。
（関数内で static 変数を用いて、２回目以降の呼び出しを検知する）

2-9-3. スタックサイズに注意！

カテゴリ	信頼性-スタックサイズ
運用基準	推奨

【詳細説明】
タスクのスタックサイズは、必要十分か(逆に不必要に確保しすぎていないか → 他のリソースに影響しないか)を確認すること。
OS の初期値をそのまま使う場合も、コメントで補足が必要。
割込み処理のスタックサイズも、オーバーフローの危険性があるため留意すべきである。
(NORTi は、スタックポインタを除く CPU の全レジスタサイズの 4 倍[byte]が標準値)

タスク内にてライブラリ等、使用スタックを見積もれない関数を使用する場合は、余裕を持って確保し、使用状況をチェックし最適化を行う。
スタックの使用状況確認方法として、タスク起動前にスタックメモリを固定値でフィルし、連続運転しながら一通りの機能を動かした後に、スタックの最大使用量を確認する。（NORTi の場合はスタック初期化時にタスク ID にてフィルされる）

【備考】
HEW、ARM の開発環境では、各関数のスタック使用領域を計測するツール(call walker)やリンカオプション(-callgraph)が存在するので、そのツールを有効活用すること。アドレス登録された関数を呼びだす場合は計測対象とならないが、使用するスタックサイズの目安になる。（実動作で計測する場合は、呼び出ししない個所で使用されるスタック使用量を計測できないので。）

2-9-4. 排他制御用資源の初期設定値	
カテゴリ	信頼性-排他制御
運用基準	遵守
【詳細説明】 排他制御を行う場合の資源番号を変数で扱う場合は、初期値を-1 などの値で初期化しておき資源番号設定忘れで使用された時にエラー状態となるようにしておくこと。 前記の対応未実施の場合は、初期化時に０がディフォルト値とされ資源占有用の関数で正常扱いとなり、すべての資源が同じ資源番号になってしまったり、途中で資源番号が変化したりしてデッドロックを発生させる可能性がある。	
【】	
【備考】	

2-10. #include

2-10-1. ヘッダーファイルは適材適所

カテゴリ	#include
運用基準	推奨

【詳細説明】

1. ヘッダーファイルを乱発しないこと
   1) 入れ子にしないこと
   2) 静的変数、関数は、実体を記載したファイルの中で定義するか、
      そのためだけのヘッダーファイルにする。

2. グローバルな宣言用のヘッダーファイルとローカル宣言用のヘッダーファイルの使い分けを明確に行うこと
   1)ローカル：　自分のモジュールでしか使わないもの
   2)グローバル：外部から利用するもの

3. グローバルなヘッダーファイル（ライブラリ等）
   プロトタイプ宣言
   使い方、入出力仕様等のコメントを記載する。

【良い例】

[a.c の中身]

```
--------------------
#include “a.h”

static      int    func(int a);     // 宣言自身は a.c の中にある

static      int    func (int a)
{
}

int    funcB (int b)
{
}
--------------------
```

[a.h の中身]

```
--------------------
extern     int    funcB(int b);   // 宣言自身は a.h の中にある
--------------------
```

【備考】

2-10-2. ファイル名は、大文字または小文字に統一

カテゴリ	#include
運用基準	推奨

【詳細説明】
ファイル名、ディレクトリ名または #include で指定するヘッダーファイル名は、大文字または小文字に統一すると、移植性が増す。また、ファイル名やディレクトリ名は小文字に統一した方が OS が代わった時の移植性が向上するので小文字を使用することを推奨する。

【良くない例】
#include "../../lib/radar_config.h"
実際のディレクトリ名：Lib

実際のディレクトリが "lib" ではなく "Lib" になっていると、
Linux のファイルシステムは殆どが大文字小文字を区別するので、../../lib が無い事になる。
Windows のファイルシステムは、大文字小文字を同一視するので、ヘッダーとして正しいパスになる。

【良い例】
#include "../../lib/radar_config.h"
実際のディレクトリ名：lib

【備考】
Linux 移植案件時は、上記の問題がないかを事前確認する。
INS グループでは、ファイル名、ディレクトリ名は全て小文字で開発を行っている。

2-11. #define

2-11-1. 配列宣言時にマジックナンバーを使うな

カテゴリ	#define-配列宣言
運用基準	遵守

【詳細説明】
例えば、配列をクリアしたい場合、配列宣言の要素数にマジックナンバー(k[3])を書き、かつ、クリアする部分に同じマジックナンバー(for(i=0; i<3;...)を書いている。#define 宣言するように変更すること。

信頼性を損なうだけでなく、配列の要素数が変更になった場合に変更箇所が増えるデメリットもある。

【良い例】

```
#define    DATA_MAX     (5)

int    L_data[DATA_MAX];
void func ( void )
{
        int    i;

        for( i=0; i<DATA_MAX; i++ ){
                L_data[i] = i;
        }
}
```

【良くない例】

```
int    L_data[5];
void func ( void )
{
        int    i;

        for(i=0; i<5; i++){
                L_data[i] = i;
        }
}
```

【備考】

2-11-2. #define 定義の注意点は、コメントに記載せよ

カテゴリ	#define
運用基準	遵守

【詳細説明】
define した値が、最初に定義した値以外に変更すると誤動作するような場合は、そのことをコメントとして記載しておくこと。（下記事例参照）
また、誤って修正した場合に意図的にコンパイルエラーが発生するようなコードを付加しておくことで事前防止にもなる。（良い例参照）

【良くない例】プログラム的に配慮がない事は、無視してください。

```
#define   BUFF_SIZE     (256)
static    int   loop_buffer[BUFF_SIZE];
static    int   w_point;

int   dataWrite ( int data )
{
    static    int   w_point=0;

    loop_buff[w_point] = data;
    w_point = (w_point+1) & 0xFF;     //書き込みポインタ更新
}
```

【良い例】

```
#define   BUFF_SIZE     (256)     //256 以外のサイズに変更する場合は、プログラムも修正すること
#if !defined(BUFF_SIZE) || BUFF_SIZE != 256
#error "BUFF_SIZE must be 256!"
#endif
static    int   loop_buffer[BUFF_SIZE];
static    int   w_point;

int   dataWrite ( int data )
{
    static    int   w_point=0;

    loop_buff[w_point] = data;
    w_point = (w_point+1) & 0xFF;     //書き込みポインタ更新(配列数 256 が前提の実装
}
```

【備考】

2-12. 保守

2-12-1. #if 0 は思い切って削除を

カテゴリ	保守
運用基準	推奨

【詳細説明】
不具合の修正等でコード削除するとき、#if 0 などで過去の実装を残しておくことは、可読性を下げ開発効率を下げる場合がある。
仕様が変更された場合、速やかに過去の実装を修正（削除）することで、実際のコードを見失わずに開発が行える。

ソースコードは、バージョン管理システム(Subversion など)で管理することにより過去ソースコードの取り出し、比較することが容易にできる。

【良くない例】

```
#if 0
    procedure1();
    procedure2();
#endif
```

このような、使わないコードを数多く残すと可読性が悪くなる。
また、Grep などの検索機能の対象となり不要な箇所が検出されることにもなる。

【備考】

2-12-2. 一時的な実装／削除の方法

カテゴリ	保守
運用基準	ノウハウ

【詳細説明】
暫定的（一時的）な実装変更のために #if 0 を置く場合、下記の様にすると良い。
但し、このようなルールはプロジェクトで統一し開発環境記述書にも記載すること。

```
#ifdef __DELETE_TEMP__ /* 2009/12/21 A.Furuno 実験のため一時的に削除 { */
    procedure1();
    procedure2();
#endif /* } */
```

"__DELETE_TEMP__" は存在しない #define とする。
この様にする事で、何カ所かを実装変更した後、やっぱり元に戻す場合、__DELETE_TEMP__ で検索すれば、戻し忘れを防ぐ事が出来る。

また、最後に '{' をコメント内に埋め込み、#endif でもコメントに '}' を埋め込む。
この様にする事で、テキストエディタの「対応する括弧の検索機能」で、どこからどこまでがコンパイルされないかも容易に検出できる。

【備考】

2-12-3. 流用コードは常に最新版を	
カテゴリ	保守
運用基準	遵守

【詳細説明】
他の開発中プロジェクトからソースコードを流用して開発する場合は、開発終了後、流用元と流用先の当該ソフトウェアが相違することがある。
このような流用のケースでは、流用する側が主導で情報交換を行い、最新版をマージし相違をなくすこと。

【備考】
ソフトウェア流用資産として登録されているソフトウェアは、モジュール申請チェックリストにて確認すること。

2-12-4. 疑わしきは使わず

カテゴリ	保守
運用基準	推奨

【詳細説明】
標準ライブラリ関数は、確かに「枯れている」が枯れ方には２通りある。
A) 十分に保守され、信頼できる。
B) 使い物にならず、誰も使わないまま放置が続いている。
上記の事に鑑み、疑わしければ使うべきではない。
標準ライブラリ関数は、コンパイラのマニュアルを調べ、または問い合わせを行うなど、確認してから使うこと。

「使えるかどうか分からないまま」使い、後で不具合の原因となるよりは、自力で作成してコードレビューで精査して不具合を防ぐ方がまだ良い。

【備考】
SDK 等に付属している標準関数は、リエントラント（スレッドセーフ）かどうかは定かではない。
加えて、iTron 環境下で使えるかどうかも分からないし、標準関数の中で何をしているか分からない（例えば、sprintf() の中で、malloc() を呼んでいる事はあり得る）ので、標準関数は、リエントラントであること、動的メモリ[1]、ストリーム[2]、時間[3]等のリソースを利用していないことを確認の上利用すること。

[1] 動的メモリ（いわゆるヒープ空間と言われるもの：malloc(),free(),realloc() 等が相当)

[2] ストリーム（FILE 構造体：fopen(),fread(),fwrite() 等が相当）

[3] 時間（time(),clock()等が相当）

2-12-5. コンパイルスイッチ名は、機能名で！

カテゴリ	保守
運用基準	推奨

【詳細説明】
コンパイルスイッチは機能名で切り分けると、ソースコードの可読性が良くなる。

【良い例】AutoIP 機能を使用する場合(USE__AUTOIP を宣言する)

```
void        initial_process ( void )
{
        .....
#ifdef      USE_AUTOIP
        init_autoip();
#endif
        .....
}
```

【備考】

2-12-6. 機種依存の実装は駄目

カテゴリ	保守
運用基準	遵守

【詳細説明】
他機種に流用できそうなソースコード(画面、キー、メニュー)には、機種依存の実装を行ってはいけない。
(ROM/RAM のアドレスを直接指定するようなコード + レジスタ設定等)。
流用元の実装が、機種依存部分とそれ以外が上手く切り分け出来ていない場合、大幅に改変することも必要である。

【良い例】

```
<device.h>
#define    RAM_START_ADDRESS   (volatile char *)0x100000;
#define    RAM_END_ADDRESS      (volatile char *)0x500000;

<test.c>
#include  “device.h”
int    test_RAM ( char *ps, char *pe )
{
.....
}
void func ( void )
{
int    result;

result = test_RAM( RAM_START_ADRESS, RAM_END_ADDRESS );
.....
}
```

【備考】
機種依存のコードはドライバ階層に含める。

第 3 章　プログラム設計時

3-1. 関数

3-1-1. 【関数切り出しの原則】関数の部品化とソースコードの再利用

カテゴリ	関数
運用基準	遵守

【詳細説明】

１） プログラム設計をする場合は、機種共通部分をなるべく大きく切り出すこと。（ライブラリ化、部品化）

２） ライブラリ化されたものには、 print 文等の処理系依存コードを含まないこと。

３） 部品化するならば、リエントラント性を考慮し、出来る限りマルチタスクで使用でき、スレッドセーフな関数とすること。

【良くない例】エラー情報を固定の変数に保存している。

```
int   divErrFlag=0;        /* calc_div 実行時のエラー情報 */

long calc_div ( long x, lon y )
{
    long ddd=0;

    errFlag = 0;
    if( y == 0 ){
        errFlag = -1;
    }
    else{
        ddd = x / y;
    }
    return( ddd );
}
```

【備考】

3-1-2. 再帰関数取扱い注意！

カテゴリ	関数
運用基準	推奨

【詳細説明】
再帰関数はプログラムがシンプルになる利点があるが、下記のことに注意する必要がある。
再帰関数を用いると、スタックオーバーフローやメモリ効率の低下が懸念される。
他の実現方法がないか十分検討してから用いる。
検証時にコードが追いづらいという問題もある。

※オーバーフロー対策として、再帰回数に上限を設ける方法があるが、上限回数に達した時の結果をどうするかの考慮は必要。

【備考】
再帰関数にする場合は、確実に処理が完了する条件を意識する必要がある。

3-2. 実数演算

3-2-1. 収束問題

カテゴリ	実数演算
運用基準	推奨

【詳細説明】
理論式が実数の収束を前提に表現されている場合、その「収束」は必ずしも保証されないことに注意する。
例えば、Newton 法で真値に収束させるアルゴリズムでは、評価関数の値は無限に小さくなることはなくループの終了条件がいつまでたっても満たさないことがある。

【例】

```
#define    EPSILON  (1.0E-10)

double     next ( double x )
{
      return( x-(x*x*x+x+1)/(3*x*x+1) );
}
double     calc_newton ( void )
{
      double     ans, ans1;

      ans = 0.0;
      ans1 = next( ans )
      while( fabs( ans – ans1 ) > EPSILON ){     //一定の値以下になった場合に演算を終了する
            ans1 = ans;
            ans = next( ans1 );
      }
      return( ans );
}
```

【備考】

3-2-2. 0（ゼロ）や微小値での割り算に注意！

カテゴリ	実数演算
運用基準	遵守

【詳細説明】
０で割る可能性や、微小値による割り算誤差の影響およびオーバーフローの発生を検討すること。

【良くない例】

```
double    calc_vector ( double course )
{
    return( 1.0 / sin( course ) );
}
```

course に 0°(実際は、0°のラジアン値)が入力されると、分母が 0.0 になり 0 除算が発生する。
また、0.0 にならなくても 0 に近い非常に小さな値の場合は割り算の結果が非常に大きな値になる。

【良い例】

```
#define   EPSILON  (1.0E-10)
double    calc_vector ( double course )
{
    double    ddd;

    ddd = sin( course );
    if( ddd < EPSILON ){        //course で判断するのではなく、分母の値(sin(course))で判断する
        ddd = 0.0;
    }
    else{
        ddd = 1.0 / ddd;
    }
    return( ddd );
}
```

【備考】

3-2-3. 【実数演算処理速度】代替案を考えよ！	
カテゴリ	実数演算
運用基準	推奨
【詳細説明】 理論式が実数表現であっても、実装は実数演算にするべきとは限らない。 処理時間の問題をとくに勘案して実装するべきである。	
【備考】	

3-2-4. 実数比較は幅を持たせること

カテゴリ	実数演算
運用基準	遵守

【詳細説明】
実数演算に対して「比較」演算したいときは、例のように幅を持たせて表現すること。

【良くない例】

```
if( fabs( x ) == 0.5 ){
    .....
}
if( y == 0.3 ){
    .....
}
```

【良い例】

```
if( fabs( x ) <= 0.5 ){
    .....
}
if( ( 0.29 < y ) && ( y < 0.31 ) ){      // ただし、境界値の扱いの不定性に注意
    .....
}
```

【備考】

3-2-5. 実数データの整数化

カテゴリ	実数演算
運用基準	推奨

【詳細説明】
処理時間などの関係で実数の変数を使うことができない場合（航法演算等）、表示桁に合わせた変数を定義することがある。

整数化のデータとして位置、船速、方位などがあり、位置については１/１００００分、船速・方位などは１０倍値などにしておくと便利な場合が多い。

方位データについては、1=1x10E-1[deg]形式で持つと便利な場合が多いし、NMEA2000 を対応するのであれば1=1x10E-4[rad]形式で持つと便利な場合もある。

また、表示単位/出力単位を変更する機能があるならば、最も高精度の単位でデータを持つなどの工夫も必要。

精度を意識すべき例を挙げる。

```
long      sougou_data;      // 方位データ、s15[deg]
long      data;
double    a;
```

【良くない例】 (計算途中で 1=0.1deg に変換されているので精度が落ちる)

```
a = (double)sougou_data * 10.0 / 65536.0;          // s15[deg]を 1=0.1[deg]へ変換
chg_10deg_to_df02( (unsigned short)a, &data );  //(1=0.1[deg]を 1x10E-4[rad]へ変換)
```

【良い例】 (s15 のまま rad へ変換する)

```
chg_s15deg_to_df02( sougou_data, &data );          //s15[deg]を 1x10E-4[rad]へ変換
```

【備考】

3-2-6. 実数精度により三角関数の計算結果が範囲を超える

カテゴリ	実数演算
運用基準	遵守

【詳細説明】
三角関数の計算結果は理論的には一定の範囲に収まるが、計算機においては、浮動小数点数等の精度の問題で範囲を超えてしまうことがある。

範囲を超えた計算結果をそのまま以降の処理に使用すると、想定外の計算結果になる可能性があるため、計算結果の範囲チェックや関数の成否確認を行うこと。

【良くない例】

```
/* fCos が 1.0f よりも大きい場合、sqrtf の引数が負値となり非数が返される */
fTan = sqrtf( ( 1.0f - fCos ) / ( 1.0f + fCos ) );
```

【良い例】

```
/* sqrtf の引数が 0.0 未満にならないことを保証する */
fTan = sqrtf( max( 0.0f, ( 1.0f - fCos ) / ( 1.0f + fCos ) ) );
```

【備考】
sqrt 関数群は，実引数が 0.0 未満の場合、errno に定義域エラー(EDOM)を格納するので、期待通りに変換できているかどうかは、errno を用いて確認できる。以下に例示する。

```
#include <stdio.h>
#include <math.h>
#include <errno.h>

int main(void)
{
	float rv = 0.0;
	errno = 0;
	rv = sqrtf(-1.0);
	if(errno == EDOM){
		printf("DOMAIN ERROR¥n");
	}
	return 0;
}
```

sqrtf 関数は C99 規格で追加されており、C99 規格では isnan マクロも追加されているので、C99 対応コンパイラであれば、errno の参照以外に戻り値を isnan マクロで評価する事も可能である。

3-3. 論理演算

3-3-1. 否定論理は減らそう

カテゴリ	論理演算
運用基準	推奨

【詳細説明】
条件判定では肯定論理のほうが理解しやすい。
否定論理で考えついたアルゴリズムも肯定論理に置き換えられないかを考えること。
（論理判定が複合文でできているときなど、簡単化に努める。
「エラーがなかったとき」というより、「正常な回答を得たとき」と関数定義や変数定義を変えて見通しよくする）
但し、否定を使用した方が簡潔に記述出来る場合の否定使用は何ら問題ない。

【良くない例】

```
if( x != a ){
    operation1();
}
else{
    operation2();
}
```

【良い例】

```
if( x == a ) {
    operation2();
}
else{
    operation1();
}
```

【備考】

3-4. 変数

3-4-1. 静的変数をなるべく減らす

カテゴリ	変数
運用基準	遵守

【詳細説明】
静的変数の代わりに auto 変数を使える場合は、静的変数を使わないこと。

【良くない例】

```
void func ( void )
{
	static int  size;

	size = GetSize();
	PutData( size );
}
```

【良い例】

```
void func (void )
{
	int    size;

	size = GetSize( );
	PutData( size );
}
```

【備考】
静的変数を使用する事で、リエントラントな関数にならずマルチタスクで使用出来なくなる。

3-4-2. 変数参照の前に値は代入されているか？

カテゴリ	変数
運用基準	遵守

【詳細説明】
オート変数の参照時に、値が不定値となるパスが存在しないことを確認すること。

【良くない例】//ans が OK でない時に、ret が未初期化になる

```
int	func ( int ans )
{
	int	ret;

	if( ans == OK ){
		ret = 0;
	}
	return( ret );
}
```

【良い例】

```
int	func ( int ans )
{
	int	ret

	ret = -1;
	if( ans == OK ){
		ret = 0;
	}
	return( ret );
}
```

【備考】
静的解析ツールで確認することも可能だが、構造体のメンバーなどの初期化忘れを検出できない場合もあるので実装者が注意して事前確認すること。

3-4-3. アドレスアラインメントに注意

カテゴリ	変数
運用基準	推奨

【詳細説明】
ライブラリ利用時に関数に与えるバッファ(ワーキングエリア)について、サイズだけでなく先頭アドレスが偶数(奇数)であることを指定するライブラリが存在する。異なる処理系においても、極力先頭アドレスが偶数(奇数)となるような回避が可能なソースコードとするのがよい。
NORTi の udp_snd_dat/udp_rcv_dat などの関数が該当する。

【良くない例】　//UINT8 の配列の場合は、先頭アドレスが奇数になる可能性がある。

```
#define   BUFFER_SIZE  (256);
UINT8     buffer[BUFFER_SIZE];
```

【良い例】　//UINT32 で宣言することにより、先頭アドレスが偶数になる。
//但し、UINT8 と同様のサイズ定義とするために、BUFFER_SIZE/sizeof(UINT32)としている。

```
#define   BUFFER_SIZE  (256)
UINT32    buffer[BUFFER_SIZE/sizeof(UINT32)];
```

【備考】

3-4-4. タスク間通信の方法

カテゴリ	変数-グローバル
運用基準	推奨

【詳細説明】
タスク間通信でグローバル変数を直接やりとりすることはやめた方がよい（排他制御を意識する必要があるため）。
可能であれば、下記【良い例】のようなSet、 Get関数を作成し、関数内で排他制御をすること。
排他制御を行う場合は、グローバル変数を直接アクセスするのではなく、関数を利用すること。(使用する側に意識させない方がよい)

【良い例】

```
void SetXXX ( XY *xy )
{
	//排他制御
		memcpy( &tskA_xy, xy, sizeof(tskA_xy) );
	//排他制御
}

void GetXXX ( XY *to )
{
	//排他制御
		memcpy( to, &tskA_xy, sizeof(XY) );
	//排他制御
}
```

【備考】
上記関数を別の排他制御期間中に使用する時は、注意が必要なので関数ヘッダーなどに注意事項を記載しておく必要がある。

3-5. 割り込み

3-5-1. 割込み禁止/許可の実装に注意

カテゴリ	割り込み
運用基準	遵守

【詳細説明】
FURUNO 共通関数の DI() / EI()は、割込みマスクレジスタを直接設定する。スタックにレジスタ情報を退避しないので、多重化された呼び出しには対応できないことに注意すること。
そのため、c_di() / c_ei()などを利用して、割込み禁止前のマスクレジスタ情報を復帰させる形の実装にすること。

【良い例】

```
void SetA ( int a )
{
	int diF;

	diF = c_di();
		funcX();
	c_ei( diF );
}
void SetB ( int b )
{
	int diF;

	diF = c_di();
		funcY();
		SetA(a);	//関数内で割り込み解除されない
		funcZ();	// funcZ()は、割り込み禁止状態で実行される
	c_ei( diF );
  }
```

【良くない例】

```
void SetA ( int a )
{
	DI();
		funcX();
	EI();
}
void SetB ( int b )
{
	DI();
		funcY();
		SetA(a);	// SetA()内で割り込みは許可状態になっている
		funcZ();	//SetB()からみると、意図しないタイミングで割り込みが許可され、funcZ()が実行される
	EI();
}
```

【備考】
割り込み禁止だけでなく、排他制御でも同様の事象が発生するので同じ対応が必要。

3-6. 拡張性

3-6-1. データベースの情報は、なるべく最後まで持ち続ける。

カテゴリ	拡張性
運用基準	推奨

【詳細説明】
データベースの情報を、なるべく最下層まで引き継ぎながら管理する。
上層のほうで、データベースの情報を削ったりした場合、下層で利用しなくてはならなくなった場合、変更が大幅になる。

//x,y は、描画用の座標でセットという事例です。

【良くない例】関数 func1 で、変数ｘしか使用しないのでｘだけを渡している。

```
void func1 ( int x )
{
	.....
	printf( “%d”, x );
	.....
}

void draw ( int x, int y )
{
	func1( x )
}
```

【良い例】座標ｘ、ｙは、一般的にセットで扱われるので詳細説明記載のように変数ｙも引き継がせている。

```
void func1 ( int x, int y )
{
	.....
	printf( “%d”, x );
	.....
}

void draw ( int x, int y )
{
	func1( x, y )
}
```

【備考】

改訂履歴

改訂番号	改訂年月日	改訂箇所（頁・行）	改　訂　内　容	改　訂　理　由	担当	承認
1	2018/2/1	全頁	ヘッダー・フッターに【古野電気秘密情報】と[FURUNO ELECTRIC CO.,LTD. CONFIDENTIAL]を追加	当社指定様式に沿った対応。	田中	開発部 2018/01/30 和田
		11 頁	誤記修正 誤：void 正：int	void 関数であるのに値を返す例であったため（誤記）		
		3 頁	改廃を追加	ソフトウェア開発課廃止に伴う対応。		
			その他、全面改訂（詳細内容は省略しています）	適切でないガイドや事例があったため	川本	
2	2020/3/18	90 頁	新規項目追加 （実数精度により三角関数の計算結果が範囲を超える）	AR-100M 振り返り分析のフィードバック対応。	開発部 '21/06/22 西山(浩) 代	開発部 2021/06/23 和田